

Prototype Design Pattern

Intent

Create new objects by **copying an existing instance** (the *prototype*) instead of constructing one from scratch. The caller says "give me another one like this" and the object clones itself — without the caller needing to know its concrete class or how expensive it is to build.

This implementation uses Java's `Cloneable` marker interface and a self-declared `clone()` method on each product.

UML class diagram

```
<<interface>> Shape (extends Cloneable)
+-----+
| +clone() : Shape |
+-----+
      ^
      | implements
      +-----+
      | Circle |
      +-----+
      | -radius |
      | -color  |
      | +clone() --+--> returns new Circle(radius,color)
      +-----+
client -> original.clone() -> independent copy
```

The players

prototype/contract/Shape	the prototype interface (declares clone())
prototype/contract/concret/Circle	a concrete prototype that copies itself
prototype/PrototypeDesignPattern	the demo — clones a Circle and prints both

The code, line by line

Shape — the prototype interface

```
public interface Shape extends Cloneable {
    Shape clone();
}
```

- **extends Cloneable** — `Cloneable` is a *marker interface*: it has no methods, it just tags a class as "allowed to be cloned." Its role is explained in *Why Cloneable* below.
- **Shape clone();** — declares the copy operation and gives it a **covariant return type** of `Shape`. `Object` already has a `protected Object clone();` redeclaring it here does two things: it makes the method **public** (so callers can invoke it) and it **narrows the return type** from `Object` to `Shape`, so callers get a `Shape` back without casting.

Circle — a concrete prototype

```
public class Circle implements Shape {

    int radius;
    String color;

    public Circle(int radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    @Override
    public Shape clone() {
        return new Circle(radius, color);
    }
}
```

- The fields `radius` and `color` hold the state that must be reproduced in the copy.
- The **constructor** is the normal way to build a `Circle` — but the point of Prototype is to *avoid* making the caller use it directly.

- `clone()` is the heart of the pattern. This implementation copies **manually**: it reads its own current `radius` and `color` and passes them into a brand-new `Circle`. The returned object is a **separate instance** with equal field values.

PrototypeDesignPattern — the demo

```
Circle originalCircle = new Circle(10, "Black");
Circle copyCircle = (Circle) originalCircle.clone();

System.out.println(originalCircle);
System.out.println(copyCircle);
```

- `originalCircle` is the prototype.
- `originalCircle.clone()` produces a **new** `Circle(10, "Black")`. The cast `(Circle)` narrows the `Shape` return back to `Circle`.
- Printing both shows **two different** `Circle@<hashcode>` lines — different hashcodes prove they are two distinct objects in memory, not the same reference. (See the note on `toString()` below.)

Why the design decisions

Why clone instead of just calling the constructor?

Prototype pays off when **construction is expensive or complicated** — the object took a database round-trip, a heavy computation, or a long configuration sequence to reach its current state. Copying an already-built instance skips all of that. It also lets code create new objects **without knowing their concrete class**: given any `Shape`, you can call `clone()` and get another one of the same kind, even if you don't know whether it's a `Circle`, `Square`, etc. The prototype instance effectively *is* the factory.

Why Cloneable ?

`Cloneable` is Java's opt-in switch for cloning. If a class calls `Object.clone()` **without** implementing `Cloneable`, the JVM throws `CloneNotSupportedException`. So the interface is a safety gate: it signals "this type consents to being cloned." This module declares `Shape` extends `Cloneable` to express that intent for the whole family.

Subtlety: this implementation does **not** actually call `super.clone()` — it copies by hand with `new Circle(...)`. Because of that, it would technically work even without `Cloneable`. Keeping `extends Cloneable` still communicates intent and keeps the door open for implementations that *do* use `Object.clone()`.

Why declare clone() public with a Shape return type?

- `Object.clone()` is `protected`, so callers in other packages couldn't invoke it. Redefining it in the interface makes it **public**.
- Returning `Shape` (not `Object`) is a **covariant return** that spares callers a cast in the common case and keeps the API type-safe.

Manual copy vs. super.clone() — and shallow vs. deep

There are two ways to implement `clone()` :

1. **`super.clone()`** — asks `Object` to make a bitwise field-by-field copy. This is a **shallow** copy: primitive fields are duplicated, but object-reference fields are **shared** (both the original and the copy point at the same nested object). Fine here because `String` is immutable, but dangerous if a field is a mutable object (a `List`, an array) — mutating it through one copy would affect the other.
2. **Manual copy (what this code does)** — `new Circle(radius, color)`. You control exactly what gets copied. For a class with mutable nested objects you would deep-copy them here (e.g. `new ArrayList<>(other.items)`), producing a **deep** copy with no shared mutable state.

This module's fields are an `int` and an immutable `String`, so a shallow copy and this manual copy are equivalent — but writing it manually makes the copying explicit and side-steps the well-known awkwardness of `Object.clone()` (checked exception, `Cloneable` gotchas, no constructor invocation).

Why does printing show two different hashcodes?

`Circle` does **not** override `toString()`, so `System.out.println` falls back to `Object.toString()`, which prints `ClassName@hexHashCode`. The two lines differ because the original and the clone are genuinely separate objects — which is the visible proof that `clone()` produced a copy, not a shared reference. (If you wanted to confirm the **values** match rather than the identities differ, you'd override `toString()` to print `radius / color` — but that would hide the "two distinct objects" demonstration.)

Execution flow (as run from `main`)

```
PrototypeDesignPattern.main
```

```
├─ new Circle(10, "Black")      build the prototype the expensive/normal way
│   └─ originalCircle
├─ originalCircle.clone()      the object copies ITSELF
│   └─ new Circle(10, "Black") a brand-new, independent instance
│       └─ copyCircle
├─ println(originalCircle) → Circle@1a2b3c (identity #1)
└─ println(copyCircle)    → Circle@4d5e6f (identity #2 – different object)
```

Relationship to the other Creational patterns

- **Factory / Abstract Factory** create objects by **choosing a class** and calling `new` inside the factory.
- **Builder** assembles one complex object **field by field**.
- **Prototype (this module)** skips construction logic entirely and **copies an existing instance**. Reach for it when you already have a fully-configured object and want another just like it, or when you want to decouple "make another one" from the concrete class.

Note: unlike the other modules here, Prototype has no `SpringApplication.run(...)` — it's a plain `main`, because cloning needs no container. It's the purest demonstration of the pattern with nothing else in the way.